

Below is your **interview project playbook**. This is designed so that every answer quietly communicates:

1. **I understand how processors work.**
2. **I know how to verify and debug designs systematically.**
3. **I enjoy solving engineering problems, not just writing code.**
4. **I am honest about what I know, and I learn quickly.**

The strategy is simple: **do not pretend you already have production CPU DV experience. Show that you have the right foundation and that you can become useful quickly.** The uploaded strategy PDF says the same thing very clearly: do not sound like you already worked on Qualcomm CPUs; sound like someone they can trust near one.

0. Your main interview positioning

Use this frame throughout the interview:

“My professional experience is mainly in ASIC validation, embedded C/C++, Python automation, protocol debug, and hardware/software interaction. My CPU-specific exposure comes from my SERV RISC-V thesis and self-driven RTL/UVM projects. I have not worked in a production CPU DV role yet, but I have the foundation: RISC-V, RTL, UVM, regression automation, waveform/debug thinking, and validation discipline. I’m looking to grow into industrial CPU DV at Qualcomm.”

That is honest, strong, and hard to attack.

Your CV supports this because your current role is ASIC validation with Python automation, embedded C/C++, UART/SPI/I²C protocol debug, oscilloscope automation, and targeted root-cause isolation. It also lists SERV RISC-V, AES UVM, Yosys GLS, and CPU performance instrumentation as your project foundation.

1. What the interviewers probably care about

The public job and profile information suggests this is not just a “can you write SystemVerilog?” interview. It is CPU design verification: architecture awareness, verification planning, debug, coverage, and working with design/architecture teams. Public Qualcomm job mirrors describe CPU/DV engineers as working on IP/block/SoC development and collaborating cross-functionally; another processor DV listing mentions owning unit/subsystem verification and working closely with architecture, design, and implementation teams.

Public snippets show Kashif Inayat as a Qualcomm Senior Engineer in a Custom CPU Team with interests in CPU microarchitecture, computer arithmetic, ML accelerators/TPUs, and power; Saad Waheed as a Staff Engineer / experienced Design Verification Engineer skilled in Verilog, SystemVerilog, UVM, C, etc.; and Xavi Salvà Grimalt as a Computer Architect at Qualcomm.

So speak to them like this:

To Kashif / Xavi:

Processor trade-offs, datapath correctness, power/area/performance, instruction behavior, microarchitecture impact.

To Saad:

Test plan, scoreboard, assertions, coverage, constrained random, debug, regression triage.

2. Your project order tomorrow

Use this order:

1. **SERV RISC-V thesis** — best match to CPU/RISC-V/microarchitecture.
2. **AES-128 UVM** — best proof of DV methodology.
3. **TinySoC** — best proof of RTL-to-GDS/system thinking.
4. **RV32I Processor** — backup for ISA/pipeline/control-unit fundamentals.

Do **not** start with TinySoC. The interview is CPU DV; SERV is your cleanest bridge.

3. Project 1 — SERV RISC-V Core / Stochastic ALU Thesis

One-line positioning

“SERV was my master’s thesis, not industry work. The value was that I modified part of a real RV32I core, thought about execution-path changes, and validated that the added computation path did not break baseline behavior.”

Your report describes a dual-mode SERV ALU enhancement for stochastic computing, using simple logic such as AND for stochastic multiplication and OR for approximate stochastic addition, with mode selection between deterministic and stochastic paths. It also reports PYNQ-Z2 FPGA implementation, low LUT/BRAM usage, and estimated 30.15 mW per core.

The six questions

1. What was the objective?

“The objective was to extend the SERV RV32I core’s ALU so it could support stochastic computing operations while preserving SERV’s compact bit-serial nature. SERV is attractive because it is extremely small, but the default ALU mainly supports basic RV32I operations. I wanted to explore whether an optional stochastic computation path could be added without destroying the low-area/low-power character of the core.”

What this communicates:

I understand processor trade-offs: area, power, datapath simplicity, ISA behavior.

2. What exactly did you contribute?

“My contribution was around the ALU/execution-side extension. I added a stochastic ALU path, introduced control selection between deterministic and stochastic behavior, and focused on validating the modified datapath. I also analyzed FPGA utilization and power to check whether the extension still made sense for a lightweight core.”

Safe detail:

“The design used simple stochastic arithmetic ideas: AND-based multiplication and OR-based approximate addition on probabilistic bitstreams. The output selection was controlled by mode signals such as stochastic mode and MAC mode.”

The report specifically lists `i_mac_mode` and `i_stochastic_mode` as new control inputs and describes multiplexer-based output selection between deterministic and stochastic results.

3. What was the hardest challenge?

“The hardest part was not the stochastic logic itself; the hard part was integration. When you add a new path inside a processor, you must make sure the original path is not disturbed. The key question was: when stochastic mode is disabled, does the core still behave like normal RV32I SERV?”

This is the right answer.

Do not make it sound like you built Oryon in your bedroom. That is how candidates summon the verification demons.

4. How did you debug it?

“I would debug it by separating the problem into layers: first verify the standalone stochastic ALU behavior, then verify mode selection, then verify the integration path. I would check whether the wrong result comes from the operation itself, the control signal, the mux selection, or the writeback path. For waveforms, I would watch operands, mode signals, selected ALU output, comparator output, and final writeback.”

Say this even if the exact old waveforms are not fresh in your head. This is a valid engineering debug method.

5. If you started again today, what would you improve?

“I would make the verification much more CPU-DV style. I would add an ISA-level reference model, random instruction streams, assertions for illegal mode combinations, coverage for mode transitions, and regression tests that prove baseline RV32I behavior is unchanged when stochastic

mode is off. I would also document exactly which instructions or custom controls activate the new path.”

This is a powerful answer because it admits the project was academic but shows you know how industry DV would improve it.

6. What did you learn?

“I learned that processor modification is not just about adding logic. A small change in an execution unit can affect control, muxing, writeback, timing, verification boundaries, and ISA transparency. That is exactly why CPU DV is interesting to me.”

That line is gold. Use it.

“How did your ALU modification work?”

Use this answer:

“At a high level, I kept the normal deterministic ALU behavior as the baseline and added an optional stochastic computation path. In stochastic mode, values are treated as probabilistic bitstreams. Multiplication can be approximated with an AND operation, because the probability of both input streams being 1 corresponds to the product of their probabilities. Addition was approximated with OR-style logic, with the known limitation that OR can saturate and is not the same as exact binary addition.

The important architectural idea was the mode selection: normal RV32I behavior should be used by default, and the stochastic result should only be selected when the relevant mode control is enabled. So from a verification perspective, I would check three things: deterministic operations still work, stochastic operations produce expected statistical behavior, and the mux/control path selects the correct result.”

This answer is technical, but still honest.

“Why did you implement it that way?”

“Because SERV is bit-serial and area-minimal, so adding a large parallel multiplier or complex accelerator would go against the design philosophy. Stochastic computing maps naturally to serial bitstreams and simple gates. Using AND/OR-style logic and shared/muxed paths allowed the design to explore new functionality without making the core unnecessarily large.”

“What was the hardest bug?”

Use a safe, realistic answer:

“The most realistic bug class in this kind of project is mode-selection or mux-path corruption. The stochastic operation may work standalone, but once integrated, the wrong path can be selected or the deterministic path can be unintentionally affected. So I would treat wrong output as a control/data-path isolation problem: check operation result first, then mux select, then writeback.”

Do **not** invent a very specific bug unless you remember it.

4. Project 2 — AES-128 UVM Verification

One-line positioning

“AES was my self-driven DV methodology project. The goal was not only to verify AES, but to practice how real verification environments are structured: sequence, driver, monitor, scoreboard, reference model, assertions, coverage, and waveform debug.”

Your GitHub repo documents a UVM 1.2 environment with `aes_test`, `aes_env`, `aes_agent`, `sequencer`, `driver`, `monitor`, `scoreboard`, and `coverage subscriber`. It also reports 561 total transactions, all passing, 0 UVM errors/fatals, and 99.82% functional coverage.

The six questions

1. What was the objective?

“The objective was to build a self-checking UVM verification environment for an AES-128 encryption core. I wanted to move beyond simple testbenches and practice reusable verification methodology.”

2. What exactly did you contribute?

“I built the verification environment: transaction item, sequences, driver, monitor, scoreboard, reference model, functional coverage, and test structure. The driver applied plaintext/key transactions to the DUT. The monitor observed completed transactions. The scoreboard compared DUT ciphertext against the reference model. Coverage tracked whether enough input categories and transaction types had been tested.”

3. What was the hardest challenge?

“The hardest challenge was making the testbench self-checking rather than manually checking waveforms. The scoreboard had to know when the output was valid, capture the right ciphertext, compare it against the expected AES result, and avoid false mismatches from intermediate internal signal toggles.”

4. How did you debug it?

“I would debug from transaction level down to waveform level. First check whether the failing transaction was generated correctly. Then check whether the driver applied it at the right cycle. Then check whether the monitor captured the output only when `res_v_o` was valid. Then compare the scoreboard expected value and actual value. Finally, use VCD/EPWave to inspect valid/data timing.”

This maps directly to your AES README, which recommends waveform signals such as `data_v_i`, `plaintext`, `key`, `res_v_o`, and `ciphertext`, and notes that output should be captured when `valid` is high.

5. If you started again today, what would you improve?

“I would add more protocol assertions, improve coverage bins around latency and reset behavior, separate the reference model more cleanly, and add negative tests such as reset during operation or invalid input-valid timing. I would also automate regressions across multiple seeds.”

6. What did you learn?

“I learned how a verification environment becomes useful when it is self-checking. The goal is not just to run a test; the goal is to catch wrong behavior, explain why it failed, and give confidence through coverage and repeatable regressions.”

“Why did your scoreboard work like that?”

Use this answer:

“The scoreboard works by comparing the DUT output against a trusted reference model at the transaction level. AES has a deterministic expected output for a given plaintext and key, so it is suitable for a bit-exact scoreboard. The monitor captures the completed transaction when the output-valid signal is asserted, then the scoreboard computes or looks up the expected ciphertext and compares actual versus expected. That avoids manual waveform checking and makes the environment self-checking.”

“Why was this assertion needed?”

Use this generic but strong answer:

“Assertions are useful for checking rules that should always hold, independent of the test sequence. For AES, useful assertions would include reset behavior, output-valid behavior, no unknown values on valid output, and stable output when valid is asserted. The reason to use assertions is that they catch protocol/control violations close to the source, while the scoreboard catches end-to-end functional mismatches.”

This shows you know the difference between **assertions** and **scoreboards**.

5. Project 3 — TinySoC

One-line positioning

“TinySoC is not my main CPU DV project. It shows that I understand the broader RTL-to-silicon path and why clean RTL, timing, reset, interfaces, and verification assumptions matter downstream.”

The TinySoC README says it is a minimal 8-bit microcontroller-style SoC on SkyWater SKY130 using LibreLane/OpenROAD/Yosys/Magic/Netgen/KLayout, with custom 8-bit CPU, RAM, memory-mapped UART, timer, and address-decoded bus. It also says the project completed full RTL-to-GDSII flow including lint, synthesis, floorplanning, placement, CTS, routing, timing optimization, DRC, LVS, and final GDS inspection.

The six questions

1. What was the objective?

“The objective was to take a small but realistic SoC from RTL to GDSII using an open-source ASIC flow. I wanted to understand what happens after RTL: synthesis, floorplanning, placement, routing, timing, DRC/LVS, and final layout inspection.”

2. What exactly did you contribute?

“I built and integrated the small SoC components: CPU-style control, RAM, UART, timer, memory-mapped bus, and top-level integration. Then I took the design through the LibreLane/OpenROAD-based physical implementation flow.”

3. What was the hardest challenge?

“The hardest challenge was moving from ‘RTL simulates’ to ‘RTL can be physically implemented.’ Implementation exposes issues like timing, area, fanout, reset structure, clocking, memory implementation, and IO constraints. It teaches you that correct RTL is necessary but not sufficient.”

4. How did you debug it?

“I debugged by reading tool reports stage by stage: lint first, then synthesis errors, then floorplan/place/route issues, then timing reports, then DRC/LVS. Instead of treating the flow as a black box, I tried to isolate which stage introduced the problem.”

5. If you started again today, what would you improve?

“I would add stronger pre-silicon verification before implementation: bus protocol checks, UART tests, timer tests, reset tests, and ideally gate-level simulation with SDF back-annotation. I would also replace simple memory structures with proper SRAM/DFFRAM macros if targeting a more realistic chip.”

The TinySoC repo itself lists future work such as adding GPIO/SPI, timer interrupts, replacing flop RAM with memory macros, running gate-level simulation with SDF, and comparing FPGA area/timing/power.

6. What did you learn?

“I learned that DV cannot be isolated from implementation reality. Bad reset assumptions, unconstrained paths, weak interface checks, or unrealistic memories can pass in RTL but cause problems later. That makes me more careful as a verification engineer.”

6. Project 4 — RV32I Processor

One-line positioning

“This was my fundamentals project. It helped me understand instruction fetch, register file, control decode, ALU, PC update, branch/jump behavior, stalls, interrupts, and exceptions.”

Your RV32I report says the processor includes Instruction Fetch, Register File, Control Unit, and ALU modules. IF fetches instructions and updates PC; RF stores 32 registers; CU decodes opcode/funct fields and generates control signals; ALU

performs arithmetic/logical operations. The GitHub README similarly describes it as a simplified RV32I processor for arithmetic, logical, memory access, and control-transfer operations.

The six questions

1. What was the objective?

“The objective was to build a simplified RV32I processor to understand the basic CPU datapath: instruction fetch, decode/control, register read/write, ALU execution, and PC update.”

2. What exactly did you contribute?

“I implemented and tested the main modules: instruction fetch, register file, control unit, ALU, and top-level integration. The processor handled arithmetic/logical operations, branches, jumps, stalls, interrupts, and exceptions in a simplified educational form.”

3. What was the hardest challenge?

“The hardest part was coordinating control and datapath behavior. For example, a branch or jump is not just a control signal; it changes PC selection, instruction flow, and downstream behavior. Similarly, a stall means PC and instruction must remain stable.”

Your report’s testbench specifically covered instruction fetch, branch handling, jump handling, interrupt handling, stall condition, and exception detection.

4. How did you debug it?

“I debugged it by checking one module at a time. First IF: is PC updating correctly? Then CU: are opcodes producing the right control signals? Then RF: are reads/writes correct? Then ALU: are results and flags correct? Finally top-level: are all signals connected correctly?”

5. If you started again today, what would you improve?

“I would make it closer to a real CPU verification project: add an instruction memory model, data memory model, ISA reference comparison, random instruction generation, assertions for x0 behavior and PC alignment, and coverage across instruction formats.”

6. What did you learn?

“I learned how a processor is a coordination problem. The ALU can be correct, but if decode, immediate generation, register selection, or PC update is wrong, the instruction still fails. That is why CPU verification needs both block-level and system-level thinking.”

7. Answers to likely direct questions

“Why did you implement SERV that way?”

“Because the design goal was low hardware overhead. SERV is already bit-serial and area-minimal, so adding a large parallel datapath would defeat the purpose. Stochastic computing maps naturally to serial bitstreams and simple logic gates, so I used an optional stochastic path selected by control signals rather than replacing the normal ALU behavior.”

“What exactly did you verify in SERV?”

“At the level of my thesis, I verified deterministic and stochastic ALU behavior, mode switching, and FPGA implementation metrics. If I were doing this in an industrial CPU DV environment, I would extend it with ISA reference checking, random instruction streams, assertions around illegal mode combinations, and coverage for instruction classes and mode transitions.”

This answer is honest and shows growth.

“What was your hardest bug?”

Use this universal answer:

“The hardest bugs in my projects were usually not syntax errors; they were integration/debug issues where the individual block looked correct but the top-level behavior was wrong. My approach is to isolate the failure: reproduce it, identify whether it is stimulus, control, datapath, timing, or checking logic, then confirm the fix with a regression.”

“Why was coverage important in AES?”

“Coverage helped answer whether I had tested enough meaningful behavior, not just whether one test passed. A passing test only proves one scenario. Coverage tells me which transaction patterns, byte values, corner cases, or directed scenarios have been exercised. But I also understand that 99% coverage does not automatically mean the design is perfect; you still need to understand what is missing and whether those bins are meaningful.”

This matches the uploaded interview strategy PDF: coverage is about asking what was observed, what is missing, and whether missing bins are meaningful.

“What is the difference between a scoreboard and an assertion?”

“A scoreboard checks end-to-end correctness by comparing actual DUT output against expected output from a reference model. An assertion checks local rules or protocol properties that should always be true. For example, in AES, the scoreboard checks ciphertext correctness. Assertions can check reset behavior, valid signal behavior, no unknown outputs when valid is high, or stable output during valid.”

“How would you verify a CPU block?”

“I would start from the microarchitecture spec and define the test plan. Then I would identify inputs, outputs, legal states, illegal states, and corner cases. I would use directed tests for known edge cases,

constrained-random tests for broader exploration, assertions for invariants, coverage to measure progress, and a scoreboard/reference model for correctness. For CPU-level verification, I would add random instruction streams, ISA reference comparison, exception/interrupt tests, memory-ordering scenarios, and regression tracking.”

8. Your “play the man” lines for each interviewer

For Kashif

“What interested me in SERV was the trade-off: how to add functionality while preserving the tiny bit-serial nature of the core. It made me think about area, power, datapath selection, and how a small execution-path change can affect architectural correctness.”

Translation:

I think in microarchitecture trade-offs.

For Saad

“The AES UVM project helped me understand verification as a structured process: stimulus generation, driver/monitor separation, scoreboard comparison, assertions, coverage, and debug from transaction logs down to waveforms.”

Translation:

I think like a DV engineer.

For Xavi

“The RV32I and SERV work made me more interested in how instruction behavior maps into datapath and control behavior. I’m especially interested in how verification can connect functional correctness with observability and performance counters.”

Translation:

I think like someone who wants to understand the architecture, not just pass tests.

9. Your 60-second self-introduction

Memorize this:

“I’m currently working in ASIC validation in Cork, where I build Python automation, embedded C/C++ firmware, and debug hardware/software interaction issues across protocols like UART, SPI, and I²C. My direct professional experience is not production CPU DV yet, but it is strongly adjacent: validation, regression automation, protocol debug, failure isolation, and hardware-level testing.

My CPU-specific experience comes from my master’s thesis and self-driven projects. In my thesis, I worked on the SERV RV32I core and explored a dual-mode ALU extension for stochastic computing, focusing on preserving baseline behavior while adding a new computation path. Separately, I built UVM verification projects to strengthen my understanding of drivers, monitors, scoreboards, assertions, coverage, and constrained-random testing.

So I would position myself as someone transitioning into CPU DV with a practical validation/debug background and hands-on RISC-V/RTL/UVM exposure. I know I’ll need to ramp into Qualcomm’s environment, but I’m confident because the fundamentals and working style are aligned.”

10. The answer style you should use tomorrow

For every technical answer, follow this structure:

Objective → Method → Debug → Learning → Improvement

Example:

“The objective was to verify AES encryption correctness. I built a UVM environment with a driver, monitor, scoreboard, reference model, and coverage. When debugging, I would trace from transaction generation to driver timing, monitor capture, scoreboard comparison, and waveform validation. What I learned is that self-checking verification is much

stronger than manually inspecting waveforms. If I improved it today, I'd add more assertions, reset/corner-case tests, and multi-seed regressions."

This makes you sound systematic.

11. Final sentence to carry into the interview

"I'm not claiming I already have production CPU DV experience. I'm showing that I understand the fundamentals, I debug systematically, and I have already built the kind of foundation Qualcomm can train into a strong CPU verification engineer."

That is the game. That is also the man.